

From: Claude <noreply@anthropic.com>

Subject: [PATCH] feat(chat): reponses rapides, position temporaire, notes vocales

Communication chat – 3 ajouts, sans rupture de l'existant :

1. Reponses rapides : chips envoyees en un tap (Je suis arrive, J'arrive, Je suis bloque, Je ne trouve pas le lieu, Attends-moi, Merci, Annuler).
2. Partage de position temporaire : duree au choix (15/30/60 min), champ "point de repere" libre adapte aux adresses imprecises, coordonnees masquees a la lecture apres expiration (fail-closed). Le message reste en base pour l'historique et les litiges.
3. Notes vocales : audio stocke HORS du message (collection chat_media), le message ne porte qu'un media_id. get_messages renvoie jusqu'a 1000 messages : inliner du base64 audio produirait des reponses de plusieurs dizaines de Mo, injouables en 3G. Acces au media controle par appartenance a la conversation (champ `participants`, car conv_id est un join "-" d'UUID donc non parsable).

CONFORMITE – ce patch active le micro :

- RECORD_AUDIO passe de blockedPermissions a permissions
- NSMicrophoneUsageDescription ajoute (iOS)
- NSPrivacyCollectedDataTypeAudioData ajoute au privacy manifest
- STORE_COMPLIANCE_CHECKLIST.md a mettre a jour (ligne "Micro | non requis")
- declaration de collecte a mettre a jour sur App Store et Play Store

PREREQUIS : npx expo install expo-audio

Fichiers touches:

```
backend/server.py
frontend/src/api.ts
frontend/app/chat/[id].tsx
frontend/app.json
frontend/plugins/with-privacy-manifest.js
frontend/src/components/VoiceNote.tsx (nouveau)
```

diff --git a/backend/server.py b/backend/server.py

--- a/backend/server.py

+++ b/backend/server.py

@@ -393,6 +393,7 @@

```
import hashlib
```

```
import re
```

```
import unicodedata
```

```

+import base64

def _safe_regex(user_input: str, max_len: int = 80) -> str:
@@ -831,8 +832,26 @@

class MessageIn(BaseModel):
- text: str
- kind: Literal["text", "image", "location"] = "text"
+ text: str = ""
+ kind: Literal["text", "image", "location", "voice"] = "text"
+ # --- Partage de position temporaire (kind == "location") ---
+ lat: Optional[float] = Field(default=None, ge=-90, le=90)
+ lng: Optional[float] = Field(default=None, ge=-180, le=180)
+ accuracy_m: Optional[float] = Field(default=None, ge=0)
+ # Point de repère libre saisi par l'utilisateur ("Devant la pharmacie",
"Maison jaune")
+ landmark: Optional[str] = Field(default=None, max_length=200)
+ # Durée de validité du partage. Expire automatiquement - 15 min par défaut, 2
h max.
+ expires_in_minutes: int = Field(default=15, ge=1, le=120)
+
+
+class VoiceMessageIn(BaseModel):
+ """Note vocale. L'audio n'est JAMAIS stocké dans le message lui-même :
+ il part dans la collection `chat_media` et le message ne porte qu'une
référence.
+ Voir _MAX_VOICE_* pour les limites."""
+ # Audio encodé en base64 (avec ou sans préfixe data:)
+ audio_b64: str = Field(min_length=16)
+ duration_ms: int = Field(ge=500, le=120_000) # 0,5 s à 2 min
+ mime: Literal["audio/m4a", "audio/mp4", "audio/aac", "audio/mpeg"] =
"audio/m4a"

class CheckoutBookingIn(BaseModel):
@@ -2839,6 +2858,35 @@
    return out

+def _redact_expired_location(msg: dict) -> dict:
+ """Masque les coordonnées d'un partage de position dont la durée est écoulée.
+
+ Le message reste dans la conversation (pour l'historique et les litiges) mais
+ lat/lng sont retirés côté lecture : plus personne ne peut re-suivre le point
+ une fois la réservation terminée. Le point de repère textuel, lui, est

```

```

conservé
+   car c'est une information volontairement descriptive et non une
géolocalisation.
+   """
+   if msg.get("kind") != "location":
+       return msg
+   exp = msg.get("expires_at")
+   if not exp:
+       return msg
+   try:
+       exp_dt = datetime.fromisoformat(exp)
+       if exp_dt.tzinfo is None:
+           # Date sans fuseau (donnée héritée) → interprétée en UTC
+           exp_dt = exp_dt.replace(tzinfo=datetime.timezone.utc)
+           expired = exp_dt <= datetime.now(datetime.timezone.utc)
+   except Exception:
+       # En cas de date illisible on masque : la confidentialité prime sur
l'affichage.
+       expired = True
+   if expired:
+       msg = {**msg, "lat": None, "lng": None, "accuracy_m": None,
"location_expired": True}
+   else:
+       msg = {**msg, "location_expired": False}
+   return msg
+
+
@api.get("/chat/{peer_id}/messages")
async def get_messages(peer_id: str, user=Depends(current_user)):
    # Si l'un des deux a bloqué l'autre, on renvoie une conversation vide
@@ -2853,7 +2901,7 @@
        {"conv_id": cid, "to_id": user["id"], "read": {"$ne": True}},
        {"$set": {"read": True}},
    )
-   return msgs
+   return [_redact_expired_location(m) for m in msgs]

@api.post("/chat/{peer_id}/messages")
@@ -2871,10 +2919,18 @@
    if not prov:
        raise HTTPException(404, "Destinataire introuvable")
        peer_name = prov.get("name")
-   if not (body.text or "").strip():
+   is_location = body.kind == "location"
+   if is_location:
+       if body.lat is None or body.lng is None:

```

```

+         raise HTTPException(400, "Position invalide : coordonnées
manquantes.")
+     elif not (body.text or "").strip():
+         raise HTTPException(400, "Message vide")
+     # === ANTI-CIRCUMVENTION FILTER === (protect marketplace revenue)
-     original_text = body.text.strip()
+     # Le point de repère est traité comme du texte utilisateur : il passe par le
+     # même filtre, sinon il deviendrait un canal libre pour glisser un numéro.
+     original_text = (body.text or "").strip()
+     if is_location and not original_text:
+         original_text = (body.landmark or "").strip() or "Position partagée"
        filtered_text, flags = _sanitize_message(original_text)
        # Log potential contournement dans une collection dédiée pour le fraud
scoring
        if flags:
@@ -2901,6 +2957,20 @@
            "read": False,
            "created_at": now_iso(),
        }
+     if is_location:
+         expires_at = datetime.now(timezone.utc) +
timedelta(minutes=body.expires_in_minutes)
+         landmark_clean, landmark_flags = _sanitize_message((body.landmark or
"".strip()) if body.landmark else ("", []))
+         doc.update({
+             "lat": body.lat,
+             "lng": body.lng,
+             "accuracy_m": body.accuracy_m,
+             "landmark": landmark_clean or None,
+             "expires_at": expires_at.isoformat(),
+             "expires_in_minutes": body.expires_in_minutes,
+         })
+         if landmark_flags:
+             doc["flagged"] = True
+             doc["flags"] = list(*doc.get("flags", []), *landmark_flags))
        await db.messages.insert_one(doc)
        # Notifier le destinataire uniquement s'il a un compte utilisateur réel
        if peer:
@@ -2926,9 +2996,158 @@
    )
    except Exception as e:
        log.warning(f"Push failed (non-blocking): {e}")
+     return _redact_expired_location({k: v for k, v in doc.items() if k != "_id"})
+
+
+# ----- Notes vocales -----
+
+# Décision d'architecture : l'audio N'EST PAS inline dans le document message.
```

```

+# `get_messages` renvoie jusqu'à 1000 messages ; si chaque note vocale portait
+# ~250 Ko de base64, une conversation active produirait une réponse de plusieurs
+# dizaines de Mo – inutilisable sur la 3G sénégalaise, et proche de la limite
+# BSON de 16 Mo par document. L'audio va donc dans `chat_media` et le message ne
+# porte qu'un `media_id` ; le client télécharge le son à la demande, au tap.
+_MAX_VOICE_BYTES = 400_000          # ~2 min d'AAC 24 kbps, marge incluse
+_MAX_VOICE_SECONDS = 120
+
+
+def _decode_voice_payload(audio_b64: str) -> bytes:
+    """Décode et valide la taille d'une note vocale base64."""
+    raw = audio_b64.strip()
+    if raw.startswith("data:"):
+        # data:audio/m4a;base64,XXXX
+        _, _, raw = raw.partition(",")
+        # Rejet précoce sur la taille encodée (évite de décoder 50 Mo pour rien).
+        if len(raw) > _MAX_VOICE_BYTES * 2:
+            raise HTTPException(413, "Note vocale trop longue (2 min maximum).")
+        try:
+            audio = base64.b64decode(raw, validate=True)
+        except Exception:
+            raise HTTPException(400, "Audio invalide.")
+        if not audio:
+            raise HTTPException(400, "Audio vide.")
+        if len(audio) > _MAX_VOICE_BYTES:
+            raise HTTPException(413, "Note vocale trop lourde (2 min maximum).")
+        return audio
+
+
+@api.post("/chat/{peer_id}/voice")
+async def send_voice_message(peer_id: str, body: VoiceMessageIn,
+user=Depends(current_user)):
+    """Envoie une note vocale. L'audio est stocké séparément, le message ne
+    contient qu'une référence – voir la note d'architecture ci-dessus."""
+    if await _is_pair_blocked(user["id"], peer_id):
+        raise HTTPException(403, "Impossible d'envoyer un message : vous ou
+l'autre partie l'avez bloqué.e.")
+
+    peer = await db.users.find_one({"id": peer_id}, {"_id": 0})
+    peer_name = None
+    if peer:
+        peer_name = peer.get("name")
+    else:
+        prov = await db.providers.find_one({"id": peer_id}, {"_id": 0})
+        if not prov:
+            raise HTTPException(404, "Destinataire introuvable")
+        peer_name = prov.get("name")

```

```

+
+ if body.duration_ms > _MAX_VOICE_SECONDS * 1000:
+     raise HTTPException(400, "Note vocale trop longue (2 min maximum).")
+
+ audio = _decode_voice_payload(body.audio_b64)
+ cid = _conv_id(user["id"], peer_id)
+ media_id = str(uuid.uuid4())
+
+ await db.chat_media.insert_one({
+     "id": media_id,
+     "conv_id": cid,
+     # Participants stockés explicitement : conv_id est un join par "-" et les
+     # ids sont des UUID (qui contiennent déjà des tirets), donc il n'est PAS
+     # parsable de façon fiable. Le contrôle d'accès s'appuie sur ce champ.
+     "participants": sorted([user["id"], peer_id]),
+     "owner_id": user["id"],
+     "kind": "voice",
+     "mime": body.mime,
+     "size_bytes": len(audio),
+     "duration_ms": body.duration_ms,
+     "data": audio, # bytes → stocké en BinData, pas en base64
+     "created_at": now_iso(),
+ })
+
+ doc = {
+     "id": str(uuid.uuid4()),
+     "conv_id": cid,
+     "from_id": user["id"],
+     "from_name": user["name"],
+     "to_id": peer_id,
+     "to_name": peer_name,
+     "text": "🗣️ Note vocale",
+     "kind": "voice",
+     "media_id": media_id,
+     "duration_ms": body.duration_ms,
+     "mime": body.mime,
+     "read": False,
+     "created_at": now_iso(),
+ }
+ await db.messages.insert_one(doc)
+
+ if peer:
+     await db.notifications.insert_one({
+         "id": str(uuid.uuid4()),
+         "user_id": peer_id,
+         "type": "message",
+         "title": user["name"],

```

```

+         "body": "🗣️ Note vocale",
+         "peer_id": user["id"],
+         "read": False,
+         "created_at": now_iso(),
+     })
+     try:
+         await send_push(
+             recipients=[peer_id],
+             data={
+                 "title": user["name"],
+                 "message": "🗣️ Note vocale",
+                 "action_url": f"/chat/{user['id']}",
+             },
+         )
+     except Exception as e:
+         log.warning(f"Push failed (non-blocking): {e}")
+
+     return {k: v for k, v in doc.items() if k != "_id"}

@api.get("/chat/media/{media_id}")
+async def get_chat_media(media_id: str, user=Depends(current_user)):
+    """Sert un média de conversation. Accès réservé aux DEUX participants :
+    l'id du média seul ne suffit pas, on revalide l'appartenance à la
+    conversation."""
+    media = await db.chat_media.find_one({"id": media_id}, {"_id": 0})
+    if not media:
+        raise HTTPException(404, "Média introuvable")
+
+    # Contrôle d'accès : l'id du média ne suffit pas, on exige l'appartenance
+    # à la conversation. On s'appuie sur `participants` (voir insert) car conv_id
+    # n'est pas parsable : c'est un join "-" d'UUID contenant eux-mêmes des
+    tirets.
+    participants = media.get("participants") or []
+    if not participants:
+        # Doc hérité sans participants → on refuse plutôt que de deviner.
+        raise HTTPException(403, "Accès refusé à ce média.")
+    if user["id"] not in participants:
+        raise HTTPException(403, "Accès refusé à ce média.")
+
+    other = next((p for p in participants if p != user["id"]), None)
+    if other and await _is_pair_blocked(user["id"], other):
+        raise HTTPException(403, "Accès refusé à ce média.")
+
+    data = media.get("data") or b""
+    if isinstance(data, str): # tolérance si un ancien doc stockait du base64
+        try:

```

```

+         data = base64.b64decode(data)
+     except Exception:
+         raise HTTPException(500, "Média illisible.")
+     return Response(
+         content=bytes(data),
+         media_type=media.get("mime") or "audio/m4a",
+         headers={"Cache-Control": "private, max-age=86400"},
+     )
+
+
+
+ # ----- marketplace: wallet, cash payment, admin stats, fraud -----
+ async def _get_wallet(uid: str) -> dict:
+     """Récupère (ou initialise) le portefeuille d'un utilisateur."""
diff --git a/frontend/app/chat/[id].tsx b/frontend/app/chat/[id].tsx
--- a/frontend/app/chat/[id].tsx
+++ b/frontend/app/chat/[id].tsx
@@ -1,5 +1,5 @@
+ import { useCallback, useEffect, useRef, useState } from "react";
- import { View, StyleSheet, FlatList, KeyboardAvoidingView, Platform, TextInput,
+ import { View, StyleSheet, FlatList, KeyboardAvoidingView, Platform, TextInput,
Pressable, Alert } from "react-native";
+import { View, StyleSheet, FlatList, KeyboardAvoidingView, Platform, TextInput,
Pressable, Alert, ScrollView, Linking } from "react-native";
+ import { SafeAreaView, useSafeAreaInsets } from "react-native-safe-area-context";
+ import { useLocalSearchParams, useRouter } from "expo-router";
+ import { Ionicons } from "@expo/vector-icons";
@@ -7,8 +7,33 @@
+ import { api, Message } from "@src/api";
+ import { Avatar, Txt } from "@src/components/ui";
+ import { ActionSheet, ConfirmDialog } from "@src/components/ActionSheet";
+import { useLocationPermission } from "@src/hooks/useLocationPermission";
+import {
+  useAudioRecorder,
+  useAudioRecorderState,
+  RecordingPresets,
+  AudioModule,
+  setAudioModeAsync,
+} from "expo-audio";
+import { VoiceNote } from "@src/components/VoiceNote";
+ import { colors, fs, radius, shadow, spacing } from "@src/theme";

+const QUICK_REPLIES: { icon: keyof typeof Ionicons.glyphMap; label: string }[] =
+ [
+   { icon: "checkmark-circle-outline", label: "Je suis arrivé" },
+   { icon: "navigate-outline", label: "J'arrive" },
+   { icon: "alert-circle-outline", label: "Je suis bloqué" },
+   { icon: "help-circle-outline", label: "Je ne trouve pas le lieu" },
+   { icon: "hourglass-outline", label: "Attends-moi" },

```



```

+ { icon: "heart-outline", label: "Merci" },
+ { icon: "close-circle-outline", label: "Annuler" },
+];
+
+const SHARE_DURATIONS = [
+ { minutes: 15, label: "15 min" },
+ { minutes: 30, label: "30 min" },
+ { minutes: 60, label: "1 heure" },
+];
+
export default function Chat() {
  const { id, name: nameParam } = useLocalSearchParams<{ id: string; name?:
string }>();
  const router = useRouter();
@@ -24,6 +49,15 @@
  const [blockedToast, setBlockedToast] = useState<string | null>(null);
  const [reportPrompt, setReportPrompt] = useState(false);
  const [reportReason, setReportReason] = useState("");
+ const [locSheet, setLocSheet] = useState(false);
+ const [landmark, setLandmark] = useState("");
+ const [sendingLoc, setSendingLoc] = useState(false);
+ const { requestLocation, status: locStatus, openSettings } =
useLocationPermission();
+ const audioRecorder = useAudioRecorder(RecordingPresets.LOW_QUALITY);
+ const recorderState = useAudioRecorderState(audioRecorder);
+ const [recording, setRecording] = useState(false);
+ const [uploadingVoice, setUploadingVoice] = useState(false);
+ const recordStartRef = useRef<number>(0);
  const listRef = useRef<FlatList>(null);

  const load = useCallback(async () => {
@@ -64,8 +98,17 @@
    return () => clearTimeout(t);
  }, [blockedToast]);

- const send = async () => {
-   const t = text.trim();
+ // Coupe l'enregistrement à la limite backend (2 min) plutôt que de laisser
+ // l'utilisateur parler pour rien et se faire rejeter à l'envoi.
+ useEffect(() => {
+   if (!recording) return;
+   const t = setTimeout(() => { stopAndSendRecording(); }, 120_000);
+   return () => clearTimeout(t);
+   // eslint-disable-next-line react-hooks/exhaustive-deps
+ }, [recording]);
+
+ const send = async (overrideText?: string) => {

```

```

+   const t = (overrideText ?? text).trim();
+   if (!t || !id || sending) return;
+   setSending(true);
+   setErr(null);
@@ -83,7 +126,7 @@
+   created_at: new Date().toISOString(),
+   } as any;
+   setItems((x) => [...x, optimistic]);
-   setText("");
+   if (overrideText === undefined) setText("");
+   setTimeout(() => listRef.current?.scrollToEnd({ animated: true })), 30);
+   try {
+       const m = await api.post<Message>(`/chat/${id}/messages`, { text: t, kind:
"text" });
@@ -92,7 +135,7 @@
+   } catch (e: any) {
+       // Retire l'optimiste et affiche l'erreur - remet le texte pour permettre
de réessayer
+       setItems((x) => x.filter((msg) => msg.id !== tempId));
-       setText(t);
+       if (overrideText === undefined) setText(t);
+       const status = e?.status;
+       let msg = "Impossible d'envoyer le message. Réessayez.";
+       if (status === 401) msg = "Session expirée. Reconnectez-vous.";
@@ -105,6 +148,136 @@
+   }
+   };

+   const sendQuickReply = (label: string) => {
+       if (sending) return;
+       send(label);
+   };
+
+   const shareLocation = async (minutes: number) => {
+       if (!id || sendingLoc) return;
+       setSendingLoc(true);
+       setErr(null);
+       try {
+           const coords = await requestLocation();
+           if (!coords) {
+               setErr(
+                   locStatus === "blocked"
+                       ? "Localisation bloquée. Activez-la dans les Réglages."
+                       : "Impossible d'obtenir votre position.",
+               );
+           }
+           return;
+       }

```

```

+     const m = await api.post<Message>(`/chat/${id}/messages`, {
+       kind: "location",
+       text: "",
+       lat: coords.latitude,
+       lng: coords.longitude,
+       accuracy_m: coords.accuracy ?? undefined,
+       landmark: landmark.trim() || undefined,
+       expires_in_minutes: minutes,
+     });
+     setItems((x: Message[]) => [...x, m]);
+     setLocSheet(false);
+     setLandmark("");
+     setTimeout(() => listRef.current?.scrollToEnd({ animated: true }), 30);
+   } catch (e: any) {
+     setErr(e?.message || "Impossible de partager votre position.");
+   } finally {
+     setSendingLoc(false);
+   }
+ };
+
+ const MAX_VOICE_MS = 120_000; // align   sur la limite backend (2 min)
+
+ const startRecording = async () => {
+   if (recording || uploadingVoice) return;
+   try {
+     const perm = await AudioModule.requestRecordingPermissionsAsync();
+     if (!perm.granted) {
+       setErr("Autorisez le micro pour envoyer une note vocale.");
+       return;
+     }
+     await setAudioModeAsync({ allowsRecording: true, playsInSilentMode: true
+   });
+     await audioRecorder.prepareToRecordAsync();
+     audioRecorder.record();
+     recordStartRef.current = Date.now();
+     setRecording(true);
+   } catch (e: any) {
+     setErr("Impossible de d  marrer l'enregistrement.");
+     setRecording(false);
+   }
+ };
+
+ const cancelRecording = async () => {
+   if (!recording) return;
+   setRecording(false);
+   try {
+     await audioRecorder.stop();

```

```

+   } catch {
+       /* rien à faire : l'enregistrement est abandonné */
+   }
+ };
+
+ const stopAndSendRecording = async () => {
+   if (!recording || !id) return;
+   setRecording(false);
+   const elapsed = Date.now() - recordStartRef.current;
+   try {
+     await audioRecorder.stop();
+     const uri = audioRecorder.uri;
+     if (!uri) {
+       setErr("Enregistrement vide.");
+       return;
+     }
+     if (elapsed < 500) {
+       setErr("Maintenez le bouton pour enregistrer.");
+       return;
+     }
+
+     setUploadingVoice(true);
+     // Lecture du fichier local → base64, sans dépendance supplémentaire.
+     const res = await fetch(uri);
+     const blob = await res.blob();
+     const b64: string = await new Promise((resolve, reject) => {
+       const reader = new FileReader();
+       reader.onerror = () => reject(new Error("read_failed"));
+       reader.onloadend = () => {
+         const r = String(reader.result || "");
+         resolve(r.includes(",") ? r.split(",")[1] : r);
+       };
+       reader.readAsDataURL(blob);
+     });
+
+     const m = await api.post<Message>(`/chat/${id}/voice`, {
+       audio_b64: b64,
+       duration_ms: Math.min(elapsed, MAX_VOICE_MS),
+       mime: Platform.OS === "ios" ? "audio/m4a" : "audio/mp4",
+     });
+     setItems((x: Message[]) => [...x, m]);
+     setTimeout(() => listRef.current?.scrollToEnd({ animated: true }), 30);
+   } catch (e: any) {
+     const status = e?.status;
+     if (status === 413) setErr("Note vocale trop longue (2 min maximum).");
+     else setErr(e?.message || "Impossible d'envoyer la note vocale.");
+   } finally {

```

```

+     setUpLoadingVoice(false);
+   }
+ };
+
+ const openInMaps = (lat: number, lng: number, label?: string | null) => {
+   const q = encodeURIComponent(label || "Point de rendez-vous");
+   const url = Platform.select({
+     ios: `maps://?ll=${lat},${lng}&q=${q}`,
+     android: `geo:${lat},${lng}?q=${lat},${lng} (${q})`,
+     default: `https://www.google.com/maps/search/?api=1&query=${lat},${lng}`,
+   });
+   Linking.openURL(url).catch(() =>
+     Linking.openURL(`https://www.google.com/maps/search/?
api=1&query=${lat},${lng}`).catch(() =>
+       setErr("Impossible d'ouvrir la carte."),
+     ),
+   );
+ };
+
+   const displayName = peerName || "Conversation";

+   const submitReport = async () => {
@@ -169,13 +342,47 @@
      data={items}
      keyExtractor={(m) => m.id}
      contentContainerStyle={{ padding: spacing.x1, gap: 8, paddingBottom: 8
  }}
-   renderItem={({ item }) => {
+   renderItem={({ item }: { item: Message }) => {
+     const mine = item.from_id === user?.id;
+     const isOptimistic = item.id.startsWith("temp-");
+     const isLoc = item.kind === "location";
+     const isVoice = item.kind === "voice" && !!item.media_id;
+     const expired = isLoc && (item.location_expired || item.lat == null);
+     return (
+       <View style={([styles.bubbleRow, { justifyContent: mine ? "flex-end"
: "flex-start" ]]}>
+         <View style={([styles.bubble, mine ? styles.bubbleMine :
styles.bubbleTheirs, isOptimistic && { opacity: 0.6 }]}>
-           <Txt color={mine ? colors.white : colors.text}>{item.text}
</Txt>
+           {isVoice ? (
+             <VoiceNote mediaId={item.media_id!} durationMs=
{item.duration_ms} mine={mine} />
+           ) : isLoc ? (
+             <Pressable
+               onPress={() => (!expired ? openInMaps(item.lat!, item.lng!,

```

```

item.landmark) : undefined))
+
+         disabled={expired}
+         testID={`chat-location-${item.id}`}
+
+     >
+
+         <View style={{ flexDirection: "row", alignItems: "center"
+ }}>
+
+             <Ionicons
+
+                 name={expired ? "location-outline" : "location"}
+
+                 size={18}
+
+                 color={mine ? colors.white : colors.turquoise}
+
+             />
+
+             <Txt weight="700" color={mine ? colors.white :
colors.text} style={{ marginLeft: 6 }}>
+
+                 {expired ? "Position expirée" : "Position partagée"}
+
+             </Txt>
+
+         </View>
+
+         {item.landmark ? (
+
+             <Txt size="sm" color={mine ? "rgba(255,255,255,0.9)" :
colors.textMuted} style={{ marginTop: 4 }}>
+
+                 {item.landmark}
+
+             </Txt>
+
+         ) : null}
+
+         <Txt size="xxs" color={mine ? "rgba(255,255,255,0.7)" :
colors.textSubtle} style={{ marginTop: 4 }}>
+
+             {expired
+
+                 ? "Le partage a pris fin"
+
+                 : `Visible ${item.expires_in_minutes ?? 15} min ·
Appuyez pour ouvrir la carte`}
+
+             </Txt>
+
+         </Pressable>
+
+     ) : (
+
+         <Txt color={mine ? colors.white : colors.text}>{item.text}
</Txt>
+
+     )}
+
+     <View style={{ flexDirection: "row", alignItems: "center",
alignSelf: "flex-end", marginTop: 4, gap: 4 }}>
+
+         <Txt size="xxs" color={mine ? "rgba(255,255,255,0.7)" :
colors.textSubtle}>
+
+             {new Date(item.created_at).toLocaleTimeString("fr-FR", {
hour: "2-digit", minute: "2-digit" })}
+
+         @@ -214,7 +421,50 @@
+
+             </View>
+
+         ) : null}
+
+
+     {recording ? (
+
+         <View style={styles.recBar} testID="chat-recording-bar">
+
+             <View style={styles.recDot} />

```

```

+         <Txt color={colors.white} weight="700" size="sm" style={{ flex: 1,
marginLeft: 8 }}>
+         Enregistrement... {Math.floor((recorderState?.durationMillis ?? 0) /
1000)}s
+         </Txt>
+         <Pressable onPress={cancelRecording} hitSlop={8} testID="chat-
recording-cancel">
+             <Txt color={colors.white} weight="700" size="sm">Annuler</Txt>
+         </Pressable>
+     </View>
+ ) : null}
+
+ <ScrollView
+     horizontal
+     showsHorizontalScrollIndicator={false}
+     contentContainerStyle={styles.quickRepliesRow}
+     keyboardShouldPersistTaps="handled"
+     testID="chat-quick-replies"
+ >
+     {QUICK_REPLIES.map((qr) => (
+         <Pressable
+             key={qr.label}
+             onPress={() => sendQuickReply(qr.label)}
+             disabled={sending}
+             style={[styles.quickReplyChip, sending && { opacity: 0.5 }]}
+             testID={`chat-quick-reply-${qr.label}`}
+         >
+             <Icons name={qr.icon} size={14} color={colors.turquoise} />
+             <Txt size="xs" weight="600" color={colors.midnight} style={{
marginLeft: 6 }}>
+                 {qr.label}
+             </Txt>
+         </Pressable>
+     ))}
+ </ScrollView>
+
+ <View style={[styles.inputBar, { paddingBottom: 8 + insets.bottom }]}>
+     <Pressable
+         onPress={() => setLocSheet(true)}
+         style={styles.locBtn}
+         testID="chat-share-location"
+         hitSlop={6}
+     >
+         <Icons name="location-outline" size={20} color={colors.turquoise}
/>
+     </Pressable>
+     <View style={styles.inputWrap}>

```

```

        <TextInput
            testID="chat-input"
@@ -225,19 +475,42 @@
            style={styles.input}
            multiline
            editable={!sending}
-            onSubmitEditing={send}
+            onSubmitEditing={() => send()}
            returnKeyType="send"
            blurOnSubmit={false}
        />
    </View>
-    <Pressable
-        onPress={send}
-        style={[styles.sendBtn, (!text.trim() || sending) && { opacity: 0.5
    ]}]
-        testID="chat-send"
-        disabled={!text.trim() || sending}
-    >
-        <Icons name={sending ? "hourglass" : "paper-plane"} size={18}
color={colors.white} />
-    </Pressable>
+    {text.trim().length > 0 ? (
+        <Pressable
+            onPress={() => send()}
+            style={[styles.sendBtn, sending && { opacity: 0.5 }]}
+            testID="chat-send"
+            disabled={sending}
+        >
+            <Icons name={sending ? "hourglass" : "paper-plane"} size={18}
color={colors.white} />
+        </Pressable>
+    ) : (
+        <Pressable
+            onPressIn={startRecording}
+            onPressOut={stopAndSendRecording}
+            delayLongPress={200}
+            style={[
+                styles.sendBtn,
+                recording && styles.micRecording,
+                uploadingVoice && { opacity: 0.5 },
+            ]}
+            testID="chat-mic"
+            disabled={uploadingVoice}
+            accessibilityRole="button"
+            accessibilityLabel="Maintenir pour enregistrer une note vocale"
+        >

```



```

+         <Ionicons
+             name={uploadingVoice ? "hourglass" : recording ? "stop" : "mic"}
+             size={20}
+             color={colors.white}
+         />
+     </Pressable>
+     )}
+ </View>
+ </KeyboardAvoidingView>

@@ -275,6 +548,55 @@
+     onClose={() => setConfirmBlock(false)}
+ />

+     {/* Feuille de partage de position temporaire */}
+     {locSheet ? (
+         <View style={styles.promptOverlay}>
+             <Pressable style={{ flex: 1 }} onPress={() => setLocSheet(false)} />
+             <View style={styles.promptCard}>
+                 <Txt size="lg" weight="700">Partager ma position</Txt>
+                 <Txt size="xs" color={colors.textMuted} style={{ marginTop: 4 }}>
+                     Votre position sera visible pendant la durée choisie, puis masquée
+                     automatiquement.
+                 </Txt>
+
+                 <Txt size="sm" weight="600" style={{ marginTop: spacing.md }}>Point
+                 de repère (optionnel)</Txt>
+
+                 <TextInput
+                     value={landmark}
+                     onChangeText={setLandmark}
+                     placeholder="Ex : maison jaune, devant la pharmacie..."
+                     placeholderTextColor={colors.textSubtle}
+                     style={styles.promptInput}
+                     maxLength={200}
+                     testID="location-landmark-input"
+                 />
+
+                 {locStatus === "blocked" ? (
+                     <Pressable onPress={openSettings} style={[styles.promptBtn,
+ styles.promptGhost, { marginTop: spacing.md }]}>
+                         <Txt weight="700" color={colors.turquoise}>Activer la
+                         localisation dans les Réglages</Txt>
+                     </Pressable>
+                 ) : (
+                     <View style={{ gap: 8, marginTop: spacing.md }}>
+                         {SHARE_DURATIONS.map((d) => (
+                             <Pressable

```

```

+             key={d.minutes}
+             onPress={() => shareLocation(d.minutes)}
+             disabled={sendingLoc}
+             style={[styles.durationBtn, sendingLoc && { opacity: 0.5 }]}
+             testID={`location-duration-${d.minutes}`}
+         >
+             <Ionicons name="time-outline" size={16} color=
{colors.turquoise} />
+             <Txt weight="700" style={{ marginLeft: 8 }}>Partager pendant
{d.label}</Txt>
+         </Pressable>
+     )))
+ </View>
+ )}
+
+     <Pressable onPress={() => setLocSheet(false)} style=
{[styles.promptBtn, styles.promptGhost, { marginTop: spacing.md }]}>
+         <Txt weight="700" color={colors.textMuted}>Annuler</Txt>
+     </Pressable>
+ </View>
+ </View>
+ ) : null}
+
+     {/* Prompt in-app pour le motif du signalement (Alert.prompt n'existe pas
sur Android/web) */}
+     {reportPrompt ? (
+         <View style={styles.promptOverlay}>
@@ -315,6 +637,40 @@
+         bubble: { maxWidth: "78%", padding: 12, borderRadius: 18 },
+         bubbleMine: { backgroundColor: colors.turquoise, borderBottomRightRadius: 4 },
+         bubbleTheirs: { backgroundColor: colors.surface, borderBottomLeftRadius: 4,
...shadow.soft },
+         micRecording: { backgroundColor: colors.danger },
+         recBar: {
+             flexDirection: "row",
+             alignItems: "center",
+             backgroundColor: colors.danger,
+             paddingHorizontal: spacing.lg,
+             paddingVertical: 10,
+             marginHorizontal: spacing.xl,
+             borderRadius: radius.md,
+             marginBottom: 6,
+         },
+         recDot: { width: 10, height: 10, borderRadius: 5, backgroundColor: colors.white
},
+         locBtn: { width: 44, height: 44, borderRadius: 22, backgroundColor:
colors.surface2, alignItems: "center", justifyContent: "center", marginRight: 8 },

```

```

+   durationBtn: {
+     flexDirection: "row",
+     alignItems: "center",
+     justifyContent: "center",
+     height: 48,
+     borderRadius: radius.md,
+     backgroundColor: colors.surface2,
+     borderWidth: 1,
+     borderColor: colors.divider,
+   },
+   quickRepliesRow: { paddingHorizontal: spacing.lg, paddingVertical: 8, gap: 8,
backgroundColor: colors.surface },
+   quickReplyChip: {
+     flexDirection: "row",
+     alignItems: "center",
+     paddingHorizontal: 12,
+     paddingVertical: 8,
+     borderRadius: radius.pill,
+     backgroundColor: colors.surface2,
+     borderWidth: 1,
+     borderColor: colors.divider,
+   },
+   inputBar: { flexDirection: "row", alignItems: "flex-end", padding: 10,
backgroundColor: colors.surface, borderTopWidth: 1, borderTopColor: colors.divider
},
+   inputWrap: { flex: 1, minHeight: 44, maxHeight: 120, borderRadius: 22,
backgroundColor: colors.surface2, paddingHorizontal: 16, justifyContent: "center"
},
+   input: { fontSize: fs.md, color: colors.text, paddingVertical: 10 },
diff --git a/frontend/app.json b/frontend/app.json
--- a/frontend/app.json
+++ b/frontend/app.json
@@ -20,6 +20,7 @@
    "NSPhotoLibraryUsageDescription": "Choisissez des photos pour votre
profil et missions.",
    "NSPhotoLibraryAddUsageDescription": "Enregistrez vos justificatifs et
reçus dans vos photos.",
    "NSLocationWhenInUseUsageDescription": "Trouvez les trajets et
prestataires près de vous.",
+    "NSMicrophoneUsageDescription": "Enregistrez des notes vocales pour
communiquer plus facilement avec vos clients et prestataires.",
    "ITSAppUsesNonExemptEncryption": false,
    "CFBundleDisplayName": "Jokoo",
    "LSApplicationQueriesSchemes": ["wave", "orangemoney", "comgooglemaps"],
@@ -44,6 +45,7 @@
    "android.permission.ACCESS_COARSE_LOCATION",
    "android.permission.ACCESS_FINE_LOCATION",

```

```

        "android.permission.CAMERA",
+       "android.permission.RECORD_AUDIO",
        "android.permission.READ_MEDIA_IMAGES",
        "android.permission.VIBRATE",
        "android.permission.POST_NOTIFICATIONS",
@@ -53,7 +55,6 @@
        "blockedPermissions": [
            "android.permission.SYSTEM_ALERT_WINDOW",
            "android.permission.ACCESS_BACKGROUND_LOCATION",
-       "android.permission.RECORD_AUDIO",
            "android.permission.READ_EXTERNAL_STORAGE",
            "android.permission.WRITE_EXTERNAL_STORAGE",
            "android.permission.READ_CONTACTS",
@@ -111,6 +112,12 @@
        }
    ],
    [
+       "expo-audio",
+       {
+           "microphonePermission": "Enregistrez des notes vocales pour communiquer
plus facilement avec vos clients et prestataires."
+       }
+   ],
+   [
        "expo-location",
        {
            "locationWhenInUsePermission": "Trouvez les trajets et prestataires
près de vous."
diff --git a/frontend/plugins/with-privacy-manifest.js b/frontend/plugins/with-
privacy-manifest.js
--- a/frontend/plugins/with-privacy-manifest.js
+++ b/frontend/plugins/with-privacy-manifest.js
@@ -124,6 +124,18 @@
        </dict>
        <dict>
            <key>NSPrivacyCollectedDataType</key>
+       <string>NSPrivacyCollectedDataTypeAudioData</string>
+       <key>NSPrivacyCollectedDataTypeLinked</key>
+       <true/>
+       <key>NSPrivacyCollectedDataTypeTracking</key>
+       <false/>
+       <key>NSPrivacyCollectedDataTypePurposes</key>
+       <array>
+           <string>NSPrivacyCollectedDataTypePurposeAppFunctionality</string>
+       </array>
+   </dict>
+   <dict>

```

```

+       <key>NSPrivacyCollectedDataType</key>
+       <string>NSPrivacyCollectedDataTypePaymentInfo</string>
+       <key>NSPrivacyCollectedDataTypeLinked</key>
+       <true/>
diff --git a/frontend/src/api.ts b/frontend/src/api.ts
--- a/frontend/src/api.ts
+++ b/frontend/src/api.ts
@@ -391,9 +391,23 @@
    to_id: string;
    to_name?: string;
    text: string;
-   kind: "text" | "image" | "location";
+   kind: "text" | "image" | "location" | "voice";
    read: boolean;
    created_at: string;
+   // --- Note vocale (kind === "voice") ---
+   /** Référence média : l'audio se télécharge via GET /chat/media/{media_id}. */
+   media_id?: string | null;
+   duration_ms?: number;
+   mime?: string;
+   // --- Partage de position temporaire (kind === "location") ---
+   lat?: number | null;
+   lng?: number | null;
+   accuracy_m?: number | null;
+   landmark?: string | null;
+   expires_at?: string | null;
+   expires_in_minutes?: number;
+   /** true quand la durée de partage est écoulée : les coordonnées ne sont plus
renvoyées. */
+   location_expired?: boolean;
  };

  export type Conversation = {
diff --git a/frontend/src/components/VoiceNote.tsx
b/frontend/src/components/VoiceNote.tsx
new file mode 100644
--- /dev/null
+++ b/frontend/src/components/VoiceNote.tsx
@@ -0,0 +1,128 @@
+import { useMemo, useState } from "react";
+import { View, Pressable, ActivityIndicator } from "react-native";
+import { Ionicons } from "@expo/vector-icons";
+import { useAudioPlayer, useAudioPlayerStatus } from "expo-audio";
+import { Txt } from "@src/components/ui";
+import { API, getAuthTokenMem } from "@src/api";
+import { colors } from "@src/theme";
+

```

```

+/**
+ * Lecture d'une note vocale.
+ *
+ * L'audio n'est pas inline dans le message : il est servi par
+ * `GET /chat/media/{media_id}`, protégé par le même Bearer token que le
+ * reste de l'API. expo-audio accepte des en-têtes sur la source, on peut
+ * donc streamer directement sans télécharger ni stocker le fichier.
+ */
+export function VoiceNote({
+  mediaId,
+  durationMs,
+  mine,
+}): {
+  mediaId: string;
+  durationMs?: number;
+  mine: boolean;
+}) {
+  const [failed, setFailed] = useState(false);
+
+  const source = useMemo(() => {
+    const token = getAuthTokenMem();
+    return {
+      uri: `${API}/chat/media/${mediaId}`,
+      headers: token ? { Authorization: `Bearer ${token}` } : undefined,
+    };
+  }, [mediaId]);
+
+  const player = useAudioPlayer(source);
+  const status = useAudioPlayerStatus(player);
+
+  const fg = mine ? colors.white : colors.turquoise;
+  const subFg = mine ? "rgba(255,255,255,0.75)" : colors.textSubtle;
+
+  const total = durationMs ?? 0;
+  const playing = status?.playing ?? false;
+  const loaded = status?.isLoading ?? false;
+
+  const toggle = () => {
+    try {
+      if (playing) {
+        player.pause();
+      } else {
+        // Relit depuis le début si la note est arrivée à son terme.
+        if (status?.didJustFinish) player.seekTo(0);
+        player.play();
+      }
+    } catch {

```

```

+     setFailed(true);
+   }
+ };
+
+ const fmt = (ms: number) => {
+   const s = Math.max(0, Math.round(ms / 1000));
+   return `${Math.floor(s / 60)}:${String(s % 60).padStart(2, "0")}`;
+ };
+
+ // Progression : barre simple, sans dépendance de graphe.
+ const progress = total > 0 && status?.currentTime != null
+   ? Math.min(1, (status.currentTime * 1000) / total)
+   : 0;
+
+ if (failed) {
+   return (
+     <View style={{ flexDirection: "row", alignItems: "center" }}>
+       <Icons name="alert-circle-outline" size={18} color={fg} />
+       <Txt size="sm" color={subFg} style={{ marginLeft: 6 }}>
+         Note vocale indisponible
+       </Txt>
+     </View>
+   );
+ }
+
+ return (
+   <View style={{ flexDirection: "row", alignItems: "center", minWidth: 160 }}>
+     <Pressable
+       onPress={toggle}
+       hitSlop={8}
+       testID={`voice-play-${mediaId}`}
+       accessibilityRole="button"
+       accessibilityLabel={playing ? "Mettre en pause" : "Écouter la note
vocale"}
+       style={{
+         width: 36,
+         height: 36,
+         borderRadius: 18,
+         alignItems: "center",
+         justifyContent: "center",
+         backgroundColor: mine ? "rgba(255,255,255,0.22)" : colors.surface2,
+       }}
+     >
+       {!!loaded ? (
+         <ActivityIndicator size="small" color={fg} />
+       ) : (
+         <Icons name={playing ? "pause" : "play"} size={18} color={fg} />

```

```

+     })
+   </Pressable>
+
+   <View style={{ flex: 1, marginLeft: 10 }}>
+     <View
+       style={{
+         height: 4,
+         borderRadius: 2,
+         backgroundColor: mine ? "rgba(255,255,255,0.28)" : colors.divider,
+         overflow: "hidden",
+       }}
+     >
+       <View
+         style={{
+           width: `${progress * 100}%`,
+           height: "100%",
+           backgroundColor: fg,
+         }}
+       />
+     </View>
+     <Txt size="xxs" color={subFg} style={{ marginTop: 4 }}>
+       {total > 0 ? fmt(total) : "Note vocale"}
+     </Txt>
+   </View>
+ </View>
+ );
+}

```